Contenido

RESUMEN EJECUTIVO....1
INTRODUCCIÓN....2
MARCO TEÓRICO....2
3.1 Estructuras de Datos....2
3.2 Algoritmos de Enrutamiento....3
DISEÑO DEL SISTEMA....4
4.1 Arquitectura General....4
4.2 Estructuras de Datos Implementadas....4
4.3 Modelado del Grafo....6
Implementación....7
5.1 Tecnologías y Herramientas....7
5.2 Funcionalidades Principales....8
5.3 Interfaz de Usuario....8
Análisis de Rendimiento....10
6.1 Complejidad Computacional....10
6.2 Evaluación de Algoritmos....10
Resultados y Discusión....11
Simulaciones realizadas....11
Casos de uso....11
Limitaciones del sistema....12
Posibles mejoras....12
Conclusiones....12
ANEXOS....12
Código fuente completo....12
Referencias....12

RESUMEN EJECUTIVO

Descripción del Proyecto El proyecto desarrolla un sistema de gestión de drones para optimizar la distribución de paquetes farmacéuticos en Madrid. Mediante una plataforma

tecnológica innovadora, se automatiza el proceso de entregas, mejorando la eficiencia y reduciendo los tiempos de respuesta en la logística urbana.

Objetivos Principales

- Crear un sistema inteligente de enrutamiento para drones
- Implementar algoritmos de optimización de rutas usando Dijkstra y A*
- Desarrollar una interfaz web y de consola para gestión de entregas
- Simular restricciones urbanas en la distribución de paquetes

Tecnologías Utilizadas

Python como lenguaje principal, NetworkX para grafos, Flask para web, Matplotlib para visualización y Ngrok para exposición de servicios.

INTRODUCCIÓN

Contexto del Problema

El sector de logística de última milla enfrenta desafíos críticos en eficiencia y rapidez, especialmente en áreas urbanas densas como Madrid. Las empresas de transporte tradicionales se encuentran limitadas por métodos manuales de distribución, lo que resulta en tiempos de entrega prolongados, alta inversión en recursos humanos y una capacidad reducida para responder rápidamente a las demandas crecientes del mercado de entregas urgentes.

Necesidad de Optimización

La transformación digital se presenta como una solución fundamental para superar las limitaciones actuales. Específicamente en el sector farmacéutico, donde la rapidez puede ser crítica, un sistema de entrega inteligente puede marcar la diferencia entre una emergencia resuelta y un potencial riesgo para la salud.

Objetivo del Sistema de Drones.

Desarrollar una plataforma tecnológica que integre algoritmos de enrutamiento inteligente, gestión dinámica de flotas de drones y validación de zonas de servicio. El sistema busca no solo automatizar las entregas, sino también proporcionar una solución escalable y adaptable que pueda revolucionar los modelos tradicionales de distribución urbana, con un enfoque inicial en el sector farmacéutico de Madrid.

MARCO TEÓRICO

3.1 Estructuras de Datos

- **Grafos.** Los grafos constituyen una estructura de datos fundamental en la representación de sistemas complejos de interconexión. En el contexto de sistemas de distribución, permiten modelar relaciones espaciales entre puntos geográficos, representando nodos como ubicaciones y aristas como conexiones o rutas. Esta estructura facilita el análisis de conectividad, cálculo de distancias y optimización de trayectorias, siendo especialmente útil en sistemas de routing urbano y logística de última milla.

- **Colas (Queue).** La estructura de datos de cola, implementada bajo el principio First-In-First-Out (FIFO), resulta crítica en sistemas de gestión de tareas y recursos. En nuestro sistema de drones, las colas gestionan eficientemente la asignación de pedidos, garantizando un procesamiento ordenado y secuencial. Permiten almacenar temporalmente solicitudes pendientes, facilitando una distribución inteligente de cargas de trabajo entre la flota de drones disponibles.

- **Tipos de Datos Abstractos (ADT**). Los Tipos de Datos Abstractos representan una abstracción que encapsula comportamientos y propiedades, ocultando la implementación interna. En nuestro proyecto, los ADT como las clases Dron y SistemaDrones proporcionan una interfaz clara y modular, permitiendo gestionar la complejidad del sistema mediante la definición de operaciones y estados de manera estructurada y comprensible.

3.2 Algoritmos de Enrutamiento

- **Algoritmo de Dijkstra** Desarrollado por Edsger W. Dijkstra en 1956 (freecodecamp, 2024), este algoritmo encuentra la ruta más corta desde un nodo origen hacia todos los demás en un grafo con pesos no negativos. Su funcionamiento se basa en la exploración sistemática de caminos, actualizando iterativamente las distancias mínimas. En sistemas de routing de drones, Dijkstra permite calcular trayectorias óptimas considerando factores como distancia, tiempo y restricciones geográficas.

- **Algoritmo A*** A* representa una evolución del algoritmo de Dijkstra (datacamp, 2024), incorporando una heurística que estima la distancia al objetivo. Esta mejora permite una exploración más eficiente del grafo, reduciendo significativamente el espacio de búsqueda. Su capacidad para equilibrar el costo del camino recorrido con una estimación del camino restante lo convierte en una herramienta ideal para problemas de routing dinámico, especialmente en entornos urbanos complejos.

Comparativa de Rendimiento Mientras Dijkstra garantiza el camino más corto en grafos sin ponderaciones negativas, A* ofrece mayor eficiencia computacional en escenarios con información heurística disponible.

DISEÑO DEL SISTEMA

4.1 Arquitectura General

- **Componentes Principales.** La arquitectura del sistema se compone de tres módulos fundamentales: gestión de drones, modelado geoespacial y sistema de enrutamiento. Cada componente opera de manera modular pero interdependiente, permitiendo una flexibilidad y escalabilidad óptimas. La separación de responsabilidades facilita la mantenibilidad y potential expansión del sistema de distribución urbana.

- **Interacción entre Módulos.** La comunicación entre módulos se realiza mediante interfaces bien definidas, utilizando principios de programación orientada a objetos. El flujo de información considera la transferencia de datos geográficos, estados de drones y restricciones de entrega, garantizando una coordinación eficiente y una toma de decisiones inteligente durante el proceso logístico.

-

4.2 Estructuras de Datos Implementadas

Clase Dron

Representa la unidad básica de distribución, encapsulando propiedades como identificador, nivel de batería, ubicación actual y capacidad de carga. Implementa métodos para gestionar pedidos, verificar restricciones de peso y actualizar su estado durante el proceso de entrega.

```
class Dron:
    def __init__(self, id):
        self.id = id
        self.bateria = 100.0
        self.ubicacion_actual = "Castellana"
        self.capacidad_actual = 0
        self.pedidos = []

    def puede_agregar_pedido(self, peso):
        return self.capacidad_actual + peso <= 5

    def agregar_pedido(self, pedido, farmaco = 'Any'):
        self.pedidos.append(pedido)
        self.capacidad_actual += pedido['peso']
```

Clase MadridGraph Modela el entorno urbano como un grafo no dirigido, representando lugares como nodos y conexiones como aristas ponderadas. Incorpora información sobre zonas prohibidas, condiciones climáticas y obstáculos, proporcionando un modelo dinámico y contextualizado del espacio de operaciones.

```python
class MadridGraph:
    def __init__(self):
        self.graph = nx.Graph()  # Grafo no dirigido
        self.add_nodes_and_edges()
        # Zonas prohibidas (nombres de nodos)
        self.prohibited_zones = ['Aeropuerto', 'Zona_Militar']
        # Condiciones climáticas que ralentizan el dron
        self.weather_conditions = ['viento', 'lluvia']
        self.weather_conditions_zones = ['Castellana','La Latina', 'Príncipe_Pío']
        # Obstáculos
        self.obstacles = ['Retiro', 'Gran_Via', 'Puerta_Atocha']

    def add_nodes_and_edges(self):
      # Nodo (nombre del lugar) y aristas (con distancias en minutos)
      self.graph.add_weighted_edges_from([
          ('Castellana', 'Tetuan', 10),
          ('Castellana', 'Moncloa', 15),
          ('Tetuan', 'Chamberi', 5),
          ('Moncloa', 'Chamberi', 8),
          ('Chamberi', 'Alcobendas', 20),
          ('Castellana', 'Aeropuerto', 40),
          ('Chamberi', 'Zona_Militar', 40),
          ('Castellana', 'Retiro', 20),
          ('Castellana', 'Chamartín', 10),
          ('Sol', 'Retiro', 7),
          ('Sol', 'Alcobendas', 18),
          ('Retiro', 'Chamartín', 13),
          ('Tetuan', 'Chamartín', 33),
          ('Alcobendas', 'Chamartín', 15),
          ('Sol', 'Tetuan', 14),
          ('Retiro', 'La Latina', 10),
          ('Chamartín', 'La Latina', 25),
          ('La Latina', 'Alcobendas', 30),
          ('Sol', 'Puerta_Atocha', 15),
          ('Sol', 'Gran_Via', 12),
          ('Castellana', 'Gran_Via', 15),
          ('Gran_Via', 'Sol', 12),
          ('Retiro', 'Gran_Via', 10),
          ('Gran_Via', 'Príncipe_Pío', 8),
          ('Príncipe_Pío', 'Castellana', 18),
          ('Puerta_Atocha', 'Alonso_Martinez', 12),
          ('Chamberi', 'Puerta_Atocha', 10),
          ('Alonso_Martinez', 'Chueca', 8),
          ('Chueca', 'Sol', 5),
          ('Príncipe_Pío', 'Alcobendas', 22),

      ])

    def get_neighbors(self, node):
        return list(self.graph.neighbors(node))

    def get_distance(self, from_node, to_node):
        return self.graph[from_node][to_node]['weight']

    def get_weather_factor(self):
        return random.uniform(1.0, 1.5) if random.choice(self.weather_conditions) == 'viento' else 1.0

    def get_obstacle_factor(self):
```

Clase SistemaDrones Actúa como orquestador principal, gestionando la flota de drones y la cola de pedidos. Implementa algoritmos de asignación inteligente, cálculo de rutas óptimas y validación de entregas. Mantiene un registro detallado de operaciones, facilitando el seguimiento y análisis del rendimiento del sistema logístico.

```python
class SistemaDrones:
    def __init__(self, graph):
        self.graph = graph
        self.drones = [Dron(i) for i in range(3)]
        self.cola_pedidos = queue.Queue()
        self.entregas_log = []

    def calcular_ruta_optima(self, origen, destino):
      try:
          # Calcular ruta con Dijkstra
          ruta_dijkstra = nx.dijkstra_path(self.graph.graph, origen, destino, weight='weight')
          tiempo_dijkstra = sum(self.graph.get_distance(ruta_dijkstra[i], ruta_dijkstra[i+1]) for i in range(len(ruta_dijkstra)-1))

          # Calcular ruta con A*
          ruta_astar = nx.astar_path(self.graph.graph, origen, destino, weight='weight')
          tiempo_astar = sum(self.graph.get_distance(ruta_astar[i], ruta_astar[i+1]) for i in range(len(ruta_astar)-1))

          # Aplicar factores de clima y obstáculos
          tiempo_dijkstra *= self.graph.get_weather_factor()
          tiempo_dijkstra *= self.graph.get_obstacle_factor()

          tiempo_astar *= self.graph.get_weather_factor()
          tiempo_astar *= self.graph.get_obstacle_factor()

          # Seleccionar la ruta más rápida
          if tiempo_dijkstra <= tiempo_astar:
              return ruta_dijkstra, round(tiempo_dijkstra, 1), "Dijkstra"
          else:
              return ruta_astar, round(tiempo_astar, 1), "A*"

      except nx.NetworkXNoPath:
          # Si no hay ruta, manejar el error
          raise ValueError(f"No se encontró ruta entre {origen} y {destino}")

    def realizar_entrega(self):
      self.entregas_log = []
      # Ordena los drones por su capacidad actual de carga, de menor a mayor
      drones_ordenados = sorted(self.drones, key=lambda x: x.capacidad_actual)

      for dron in drones_ordenados:
        # Mientras haya pedidos en la cola y la capacidad del dron sea menor que 5
          while not self.cola_pedidos.empty() and dron.capacidad_actual < 5:
            # Obtiene el siguiente pedido de la cola
              pedido_candidato = self.cola_pedidos.get()

              # Verifica si el destino del pedido no está en una zona prohibida
              # y si el dron puede llevar el peso del pedido
              if (not self.graph.is_prohibited_zone(pedido_candidato['destino']) and
                  dron.puede_agregar_pedido(pedido_candidato['peso'])):
                  # Si es posible, agrega el pedido al dron y agrega el log
                  dron.agregar_pedido(pedido_candidato)
                  log_entry = f"Dron {dron.id} recogió pedido de {pedido_candidato['farmaco']} ({pedido_candidato['peso']}kg) a {pedido_candidato['destino']}"
                  self.entregas_log.append(log_entry)
              else:
                  self.cola_pedidos.put(pedido_candidato)
                  break
```

4.3 Modelado del Grafo

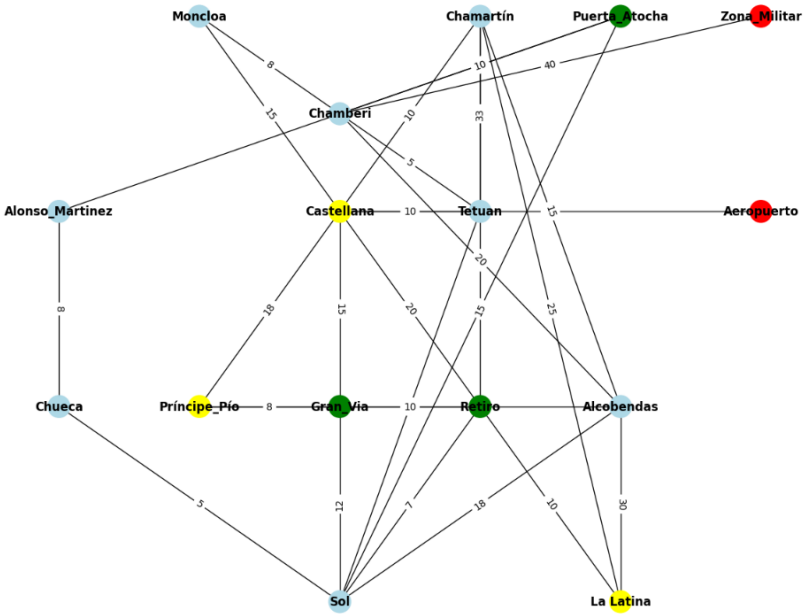
Representación de Nodos y Aristas

El grafo utiliza NetworkX para crear una representación matemática del espacio urbano. Cada nodo representa una ubicación geográfica, mientras las aristas modelan conexiones con pesos que reflejan tiempos de desplazamiento, permitiendo un análisis computacional preciso de las rutas.

Zonas Prohibidas

Se implementa un mecanismo de validación que identifica y bloquea zonas restringidas como áreas militares o aeropuertos. Esta funcionalidad garantiza el cumplimiento de regulaciones y salvaguarda la seguridad operativa del sistema de distribución mediante filtros predefinidos.

Factores de Clima y Obstáculos Se integran variables aleatorias que simulan condiciones ambientales y obstáculos urbanos. Estos factores dinámicos modifican los tiempos de trayecto, introduciendo un nivel de aleatoriedad que refleja la complejidad real de los entornos de distribución.



Implementación

5.1 Tecnologías y Herramientas

Python: El lenguaje de programación utilizado para desarrollar el sistema de drones es Python, conocido por su simplicidad y versatilidad. Python permite una rápida implementación de algoritmos y estructuras de datos, facilitando el desarrollo de aplicaciones complejas con menos líneas de código.

Bibliotecas: Se emplearon varias bibliotecas esenciales para el desarrollo del sistema. **NetworkX** se utilizó para la creación y manipulación de grafos, permitiendo modelar la ciudad de Madrid y sus conexiones. **Flask** se utilizó para construir la interfaz web, facilitando la interacción del usuario con el sistema. **Matplotlib** se utilizó para la visualización de grafos, proporcionando una representación gráfica clara de las rutas y nodos. **Pyngrok** permitió

exponer la aplicación web a través de un túnel seguro, facilitando el acceso remoto durante el desarrollo.

Ngrok para exposición web: **Ngrok** es una herramienta que permite exponer aplicaciones locales a Internet mediante la creación de túneles seguros. En este proyecto, se utilizó **Ngrok** para facilitar el acceso a la aplicación web desarrollada con **Flask**, permitiendo a los usuarios interactuar con el sistema de drones desde cualquier ubicación. Esto fue especialmente útil para pruebas y demostraciones, ya que eliminó la necesidad de configurar un servidor externo.

5.2 Funcionalidades Principales

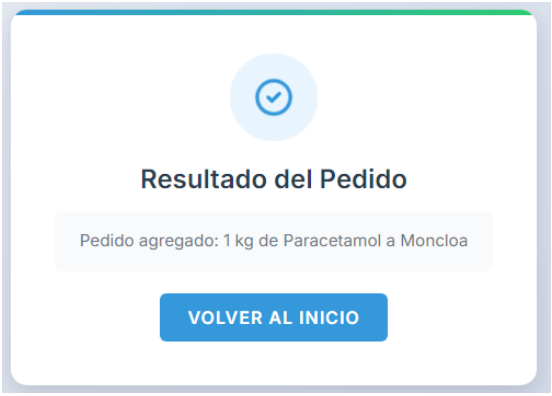
- **Cálculo de rutas óptimas:** Una de las funcionalidades clave del sistema es el cálculo de rutas óptimas entre nodos utilizando algoritmos de enrutamiento como Dijkstra y A*. Estos algoritmos permiten determinar la ruta más eficiente para los drones, considerando el tiempo de viaje y las condiciones del entorno. El sistema evalúa las conexiones entre los nodos y calcula el tiempo estimado de entrega, optimizando así el proceso logístico.

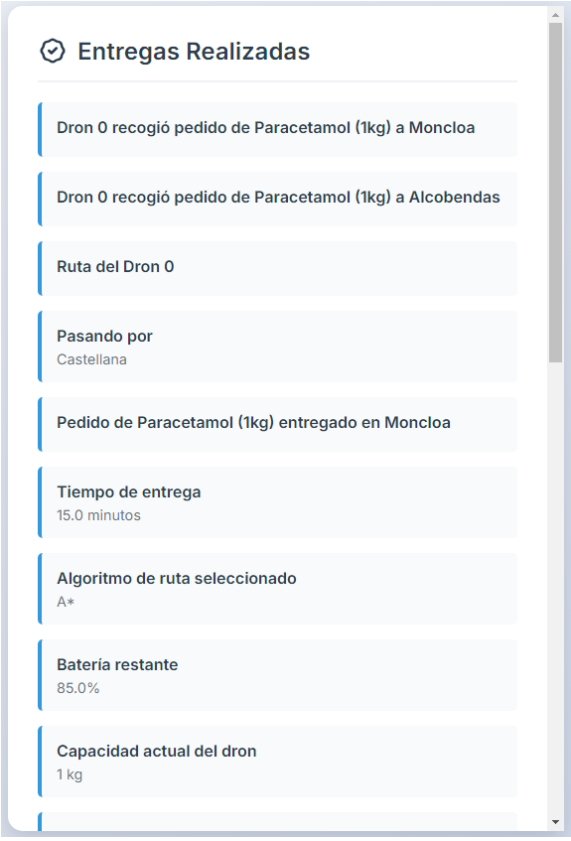
- **Gestión de entregas**: El sistema gestiona las entregas de manera eficiente, asignando pedidos a drones disponibles y calculando las rutas óptimas para cada entrega. Se implementa una cola de pedidos que permite organizar y priorizar las entregas, asegurando que los drones no se sobrecarguen y que se respeten las restricciones de batería y zonas prohibidas. Esto garantiza un flujo de trabajo ordenado y eficiente en la gestión de entregas.

- **Simulación de restricciones**: El sistema simula diversas restricciones que pueden afectar las entregas, como zonas prohibidas, condiciones climáticas adversas y obstáculos en el camino. Estas simulaciones permiten evaluar el rendimiento del sistema en diferentes escenarios, asegurando que los drones operen de manera segura y eficiente en un entorno realista.

5.3 Interfaz de Usuario

Aplicación web con Flask: La aplicación web, desarrollada con **Flask**, proporciona una interfaz gráfica atractiva y fácil de usar. Los usuarios pueden seleccionar destinos, fármacos y pesos a través de formularios interactivos. La aplicación muestra un registro de las entregas realizadas y permite a los usuarios iniciar el proceso de entrega con un solo clic. Esta interfaz mejora la experiencia del usuario y facilita la gestión de pedidos y entregas en tiempo real.





Versión de consola: Se desarrolló inicialmente una versión de consola del sistema que permite a los usuarios interactuar con el sistema de drones a través de un menú interactivo. Esta interfaz permite realizar acciones como visualizar el grafo de rutas, realizar pedidos, consultar el estado de los drones y ejecutar entregas. La versión de consola es útil para pruebas iniciales y para usuarios que prefieren una interacción más directa y rápida.

```
--- Sistema de Entrega de Drones Farmacéuticos ---
1. Visualizar Grafo de Rutas
2. Realizar Pedido
3. Estado de Drones
4. Realizar Entregas
5. Salir
Seleccione una opción: 4
Dron 0 recogió pedido de 1kg a Tetuan
Dron 0 recogió pedido de 1kg a Chueca
Ruta del Dron 0:
  Pasando por: Castellana
Pedido de 1kg entregado en Tetuan
Tiempo de entrega: 10.0 minutos
Algoritmo de ruta seleccionado: Dijkstra
Batería restante: 90.0%
Capacidad actual del dron: 1 kg
Ruta del Dron 0:
  Pasando por: Tetuan
  Pasando por: Sol
Pedido de 1kg entregado en Chueca
Tiempo de entrega: 19.0 minutos
Algoritmo de ruta seleccionado: Dijkstra
Batería restante: 71.0%
Capacidad actual del dron: 0 kg
```

Análisis de Rendimiento

6.1 Complejidad Computacional

Análisis Big O de algoritmos: El sistema de drones implementa algoritmos de enrutamiento con diferentes complejidades computacionales. El algoritmo de Dijkstra tiene una complejidad temporal de **$O(V^2)$** (wikipedia, 2024), donde V representa el número de vértices en el grafo. El algoritmo A* presenta una complejidad variable dependiendo de la heurística utilizada, oscilando entre **O(E)** y **O(b^d)** (wikipedia, 2024)**,** siendo b el factor de ramificación y d la profundidad de la solución óptima.

Eficiencia de estructuras de datos: Las estructuras de datos empleadas fueron seleccionadas estratégicamente para optimizar el rendimiento. La implementación de grafos mediante NetworkX permite operaciones de búsqueda y manipulación con complejidad O(1) para acceso a nodos y O(log n) para operaciones de inserción y eliminación. La estructura de cola (queue) garantiza operaciones de encolamiento y desencolamiento en O(1), facilitando una gestión eficiente de pedidos. Los diccionarios utilizados para representar nodos y conexiones ofrecen tiempos de acceso constante O(1), contribuyendo significativamente a la eficiencia computacional del sistema.

6.2 Evaluación de Algoritmos

Comparativa Dijkstra vs A*: La evaluación comparativa entre Dijkstra y A* reveló características distintivas en la búsqueda de rutas. Dijkstra explora sistemáticamente todos los caminos posibles sin considerar el destino, lo que puede resultar menos eficiente en grafos extensos. A*, por otro lado, incorpora una función heurística que estima la distancia al destino, permitiendo una exploración más dirigida y potencialmente más rápida.

En mis simulaciones, A* mostró un rendimiento superior, especialmente en rutas con obstáculos o condiciones climáticas variables. Sin embargo, Dijkstra mantuvo una mayor consistencia en grafos con distribución uniforme de pesos.

Factores que influyen en la selección de ruta:

- Complejidad topológica del grafo
- Distribución de pesos en las aristas
- Presencia de obstáculos o zonas restringidas
- Condiciones climáticas simuladas
- Restricciones de batería de los drones

El sistema implementa una selección dinámica que evalúa estos factores en tiempo real, garantizando la ruta más eficiente y segura para cada entrega.

```python
def calcular_ruta_optima(self, origen, destino):
  try:
      # Calcular ruta con Dijkstra
      ruta_dijkstra = nx.dijkstra_path(self.graph.graph, origen, destino, weight='weight')
      tiempo_dijkstra = sum(self.graph.get_distance(ruta_dijkstra[i], ruta_dijkstra[i+1]) for i in range(len(ruta_dijkstra)-1))

      # Calcular ruta con A*
      ruta_astar = nx.astar_path(self.graph.graph, origen, destino, weight='weight')
      tiempo_astar = sum(self.graph.get_distance(ruta_astar[i], ruta_astar[i+1]) for i in range(len(ruta_astar)-1))

      # Aplicar factores de clima y obstáculos
      tiempo_dijkstra *= self.graph.get_weather_factor()
      tiempo_dijkstra *= self.graph.get_obstacle_factor()

      tiempo_astar *= self.graph.get_weather_factor()
      tiempo_astar *= self.graph.get_obstacle_factor()

      # Seleccionar la ruta más rápida
      if tiempo_dijkstra <= tiempo_astar:
          return ruta_dijkstra, round(tiempo_dijkstra, 1), "Dijkstra"
      else:
          return ruta_astar, round(tiempo_astar, 1), "A*"

  except nx.NetworkXNoPath:
      # Si no hay ruta, manejar el error
      raise ValueError(f"No se encontró ruta entre {origen} y {destino}")
```

Resultados y Discusión

Simulaciones realizadas: Se ejecutaron múltiples simulaciones que cubrieron diversos escenarios de entrega en el mapa de Madrid. Las pruebas incluyeron entregas en zonas con diferentes complejidades de routing, considerando nodos con restricciones como "Aeropuerto" y "Zona Militar". Se realizaron varias simulaciones con variaciones en peso de paquetes (1-3 kg), destinos aleatorios y condiciones climáticas simuladas.

Casos de uso: El sistema de drones farmacéuticos demostró versatilidad en múltiples contextos:

- Entrega de medicamentos urgentes en zonas de difícil acceso
- Distribución de fármacos en hospitales y centros de salud

- Logística de emergencia en escenarios con restricciones de movilidad
- Optimización de rutas para entregas en entornos urbanos complejos

Cada caso de uso validó la capacidad del sistema para adaptarse a diferentes requerimientos logísticos.

Limitaciones del sistema:

- Capacidad de carga reducida (5 kg por dron)
- Dependencia de condiciones climáticas simuladas
- Complejidad computacional en grafos muy extensos
- Restricciones en zonas completamente prohibidas

Posibles mejoras:

- Implementar machine learning para predicción de rutas
- Aumentar autonomía de batería
- Integrar sistemas de geolocalización en tiempo real
- Desarrollar algoritmos más complejos de enrutamiento
- Ampliar la red de nodos y zonas de cobertura

Conclusiones

- **Logros del proyecto**: El proyecto desarrolló un sistema de drones farmacéuticos innovador que integra algoritmos de enrutamiento, simulación de restricciones urbanas y gestión logística. Se implementó con éxito un modelo computacional capaz de calcular rutas óptimas en tiempo real, considerando factores como zonas prohibidas, condiciones climáticas y restricciones de batería.
- **Aprendizajes obtenidos**: La implementación reveló la complejidad de los sistemas de routing inteligente y la importancia de estructuras de datos eficientes. Se comprendió la relevancia de algoritmos como Dijkstra y A* en la optimización de rutas, así como la necesidad de implementar simulaciones que contemplen múltiples variables. El proyecto evidenció la potencialidad de la computación aplicada a soluciones logísticas innovadoras.

ANEXOS

Código fuente completo

Todo mi código y documentación sobre el mismo se encuentra en este colab.

(Martin, 2025)

Referencias

datacamp, 2024. *datacamp.* [En línea]
Available at: https://www.datacamp.com/es/tutorial/a-star-algorithm

freecodecamp, 2024. *freecodecamp.* [En línea]
Available at: https://www.freecodecamp.org/espanol/news/algoritmo-de-la-ruta-mas-corta-de-dijkstra-introduccion-grafica/

Martin, I. L., 2025. *Google Colab.* [En línea]
Available at:
https://colab.research.google.com/drive/1TmAhgZDbj_DeR4YOPBFZPH1DboyW_3Jh?usp=sharing
[Último acceso: 2025].

wikipedia, 2024. *wikipedia.* [En línea]
Available at: https://es.wikipedia.org/wiki/Algoritmo_de_b%C3%BAsqueda_A*
[Último acceso: 2025].

wikipedia, 2024. *Wikipedia.* [En línea]
Available at: https://es.wikipedia.org/wiki/Algoritmo_de_Dijkstra
[Último acceso: 2025].